Study guide v2: finding communities in groups of people

Read sections 1 and 2 out loud twice. Then skim section 7 (questions) once. Everything else is for reference.

Words used here: a **meeting** is a hyperedge, the set of everyone who was in one group event. A **grouping** is a clustering, a way of sorting people into groups. **Community meetings** are meetings drawn from inside one planted group. **Noise meetings** are meetings drawn at random from everyone.

---

1. The one-minute version

Say this without notes.

1. Real groups are meetings. A committee is one meeting with many people, not a pile of pairs.
2. The usual shortcut turns every meeting into pairs. That forgets the pairs came from one event, and it makes big meetings count far too much.
3. Kovacs, Benedek and Palla (2025) find communities by chaining meetings that overlap enough. They give two versions of "enough".
4. Their method has four settings, and the paper leaves them to the user. It tests only an easy case, where community meetings and noise meetings have different sizes.
5. We rebuilt the method and checked it in the paper's setting. It recovers the planted groups exactly when the right version is used.
6. We added a rule that picks the settings without any labels.
7. The rule scores 0.555 and 0.586 for the two versions. The paper's default scores 0.292 and 0.229. Cheating with the answer key scores 0.644 and 0.698.
8. So the rule beats the default by a wide margin and still trails the cheat.
9. On real committee data, the rule gives no gain. We report that as a limit.

2. The story in five beats

Beat 1. Groups are meetings. Asha, Ben, Chen and Dev sit in one study session. That is one event with four people. Turning it into six pairs loses the fact that it was one event.

Beat 2. The paper chains meetings. Start with one meeting. Add any meeting that overlaps it enough. Keep going. Everyone in the final pile is one community. Two versions of "enough": one for when big meetings matter (large-edge variant), one for when huge meetings are meaningless (small-edge variant).

Beat 3. The paper leaves a gap. The four settings are left to the user, and the test is easy: size alone tells community meetings from noise meetings. A real user has no answer key and no such luck.

Beat 4. We rebuilt it and checked it (E1). In the paper's own setting the matching version scores a perfect 1.000 at every noise level. The wrong version falls apart. This tells you our code does what the paper says.

Beat 5. We added a rule for the settings (E3). It needs no labels. It beats the default and trails the oracle. This is our contribution.

Two more studies are backup. E2 asks what happens when the size cut is not clean. E4 tries real committee data.

---

3. Concepts in plain words

Hyperedge (meeting) and size

A meeting is the set of everyone who attended. Its size is the head count. Example: Asha, Ben, Chen and Dev in one session is a meeting of size 4. Asha and Ben having coffee is a meeting of size 2, which is an ordinary edge. Say: "A graph edge joins two people. A meeting joins any number."

Flattening and what it loses

Flattening replaces each meeting by every pair inside it. The session of four

becomes six pairs: AB, AC, AD, BC, BD, CD. A meeting of size s gives s(s-1)/2 pairs. Two losses. Big meetings count too much: size 8 gives 28 pairs, size 3 gives 3. And the pairs forget they came from one event: six pairs could just as well be six separate coffees. Say: "Flattening turns one event into many pairs and forgets they belong together."

Community

A group of people who meet together a lot, and meet outsiders less. Example: the chess club is a community. Its members share many meetings with each other and few with the football team.

Partition versus cover

A partition puts everyone in exactly one group. A cover lets groups overlap and lets some people be in none. Example with Asha, Ben, Chen, Dev, Eve. Partition: {Asha, Ben} and {Chen, Dev, Eve}. Cover: {Asha, Ben, Chen} and {Chen, Dev}, with Eve in none. Chen is in two groups. Say: "Percolation gives a cover, so overlap and left-out people are possible."

Clique percolation in ordinary graphs

Take a graph. A k-clique is k people who all know each other. For k = 3 that is a triangle. Two cliques are neighbours if they share k-1 people. Roll across neighbours. Everyone touched is one community. Example, k = 3. Triangle Asha-Ben-Chen and triangle Ben-Chen-Dev share two people (k-1), so they join. The community is {Asha, Ben, Chen, Dev}. Triangle Dev-Eve-Fay shares only Dev, so it stays apart. People in no triangle are left out. Say: "Roll a clique across neighbours that share all but one person."

The paper's recipe (five steps)

1. Start from one meeting.
2. Add any other meeting that overlaps it enough.
3. Add any meeting that overlaps the growing pile enough.
4. When nothing more joins, the people in the pile are one community.
5. Start again from a meeting not yet used.

The communities come from this rule, nothing else. The paper does not guess or train anything. The only open question is what "enough" means. That is where the settings come in.

Large-edge variant (settings k and I_abs)

Count only meetings with at least k people. Two meetings connect if they share at least I_abs people. Use it when big meetings are the real groups and small ones are chatter. Example: k = 3, I_abs = 2. {Asha, Ben, Chen} and {Ben, Chen, Dev} share two people, so they join. {Dev, Eve, Fay} shares only Dev, so it stays out. A pair {Asha, Ben} is dropped, since it is smaller than 3. Say: "Throw away small meetings, then chain big meetings that share enough people."

Small-edge variant (settings K and I_rel)

Count only meetings with at most K people. A meeting joins the pile if at least a fraction I_rel of its people are already in the pile. Use it when huge meetings, like a school assembly, mean nothing. Example: K = 4, I_rel = 0.5. The pile is {Asha, Ben, Chen, Dev}. The meeting {Chen, Dev, Eve, Fay} is half inside, so it joins and Eve and Fay enter. The meeting {Dev, Xan, Yu, Zed} is a quarter inside, so it stays out. Say: "Throw away huge meetings, then let a meeting join when half or more of it is already in."

Why size-2 meetings are dropped on real data: a pair with one person in the pile is already half inside, so it always joins. Pairs would glue everything together. The paper drops them, so do we.

The test bed

We invent 500 people in 10 planted groups of 50. We make two kinds of meetings: - Community meetings. Everyone comes from one group. Example: five people, all from group 3. - Noise meetings. People are drawn at random from all 500, so they almost always mix groups. Their job is to trick the method into gluing groups together.

The noise fraction p sets how many noise meetings exist: p times 500 per noise size. Worked example at p = 0.4: 200 noise meetings per size, three sizes, so 600 noise meetings. The community meetings are 10 groups times 50 meetings times 3 sizes, so 1500. At p = 0.4 about 3 meetings in 10 are noise.

Say: "Noise means meetings that mix groups at random, built to confuse the method."

Clean size cut versus partial versus full overlap

"Size cut" means: can you tell community meetings from noise meetings by size alone? - Clean cut. Community meetings are sizes 6 to 8, noise meetings are 3 to 5 (or the mirror). Keep only meetings of size 6 or more and every noise meeting is gone. This is the paper's test. - Partial overlap. Community 5 to 7, noise 4 to 6. Sizes 5 and 6 appear in both, but size 7 is pure community. One pure size is enough to filter on. - Full overlap. Both 4 to 6. Size tells you nothing.

"The cut is not clean" means some sizes are shared, so no size filter removes all noise.

ECS (the score)

ECS compares two groupings of the same people and gives a number from 0 to 1. Here the two are the planted truth and what the method found. 1 means identical. Near 0 means unrelated.

How it works, person by person. For Asha, list who she is grouped with in the truth. Then list who she is grouped with in the result. If the lists agree, Asha scores high. Average over all people.

Two details. People the method left out count as a group of one. Groups may overlap. That is why we need ECS: plain accuracy cannot handle either.

Toy fact: 100 people, truth is two perfect halves of 50, result is everyone alone. ECS is about 0.02. Almost nobody shares a group with the same people.

Say: "For every person, ECS asks whether their circle of group-mates is the same in both groupings."

Coverage

The fraction of people placed in any group at all. If 450 of 500 are in a group, coverage is 0.9.

Oracle, default and rule

- Oracle: the settings chosen by looking at the answer key. A cheat, so

an upper bound.

- Default: the settings the paper suggests with no tuning.
- Rule: our pick, made without labels.

---

4. The two experiments we present

E1: does our code do what the paper says? (30 runs per cell)

How to read the heat map. Each small square is one pair of settings. Rows are the intersection setting (I_abs or I_rel). Columns are k or K. Dark means good ECS. The top row is noise 0 and the bottom row is noise 0.4. A good setting shows as a dark patch that survives down to the bottom.

Large-edge communities (community meetings 6 to 8, noise 3 to 5):

noise	**large-edge variant**	**small-edge variant**
0	1.000	1.000
0.1	1.000	0.711
0.2	1.000	0.571
0.3	1.000	0.482
0.4	1.000	0.424

Mirror case (community meetings 3 to 5, noise 6 to 8):

noise	large-edge variant	small-edge variant
0	1.000	1.000
0.1	0.400	1.000
0.2	0.282	1.000
0.3	0.233	1.000
0.4	0.206	1.000

Both versions are exact at noise 0.

One detail the paper also notes: in the large-edge variant, I_abs of 4 or more breaks groups apart (ECS 0.86 to 0.91, even at noise 0). Two meetings of size 6 to 8 rarely share 4 people, so the chain snaps.

Say: "The right version stays perfect. The wrong version collapses. That is the paper's claim, and our code shows it."

E3: a rule for the settings, without labels

The rule in five steps. 1. Try every setting on a grid. 2. For each, look at the group sizes it produced. 3. Throw out a setting if it gives fewer than two groups, or if the biggest group is more than twice the second biggest. That

means one group swallowed the rest. 4. Score the survivors by concentration: are the groups few and large? (Each person splits their weight equally over their groups. Concentration is the sum of squared group sizes divided by n squared.) 5. Pick the top score. If nothing survives, use the paper's default.

Worked toy. 100 people, truth is two groups of 50. - A loose setting gives one group of 100. Fewer than two groups, so it is rejected. - A strict setting gives about 40 tiny groups. They pass, but concentration is about 0.03. - The middle setting gives two groups of 50. Concentration is (50x50 + 50x50) / 100x100 = 0.5. It wins, and it is the true answer.

Why the first version failed. The first version picked the highest coverage. But a bad setting can give many tiny overlapping groups that together touch everyone, so coverage reads near 1. Coverage rewards fragments. Concentration rewards a few large groups.

Numbers (1050 hypergraphs per variant, averaged over all layouts and noise levels):

	rule	**coverage rule**	**oracle**	**default**
large	0.55	0.54	0.64	0.29

	rule	coverage rule	oracle	default
large-edge	5	7	4	2
small-edge	0.586	0.544	0.698	0.229

Where it works. When the variant matches a layout with one pure meeting size, the rule is near the oracle (rule against oracle): 0.936 vs 1.000, 0.983 vs 0.999, 1.000 vs 1.000, 0.974 vs 0.974.

Where it does not. - Wrong variant for the layout: 0.263 vs 0.548, 0.286 vs 0.493, 0.233 vs 0.351, 0.226 vs 0.399. - Full overlap: 0.396 vs 0.471 (large-edge) and 0.410 vs 0.476 (small-edge).

Say: "The rule picks settings without labels. It more than doubles the default, and it still trails the cheat."

---

5. Backup experiments

E2: what if the size cut is not clean? Each method is scored at its best settings per cell. - Clean cut: the matching variant stays at 1.000 up to noise 0.8. - Partial overlap: community 5-7 vs noise 4-6 gives 0.999 for the large-edge variant. Community 4-6 vs noise 5-7 gives about 0.97 for the small-edge variant. One pure size is enough. - Full overlap, 4-6 vs 4-6: collapses to 0.601 at noise 0.1 and about 0.24 at noise 0.8, for both variants. - Clique percolation on the flattened graph timed out or ran out of memory in 278 of 350 runs. Where it finished under noise, it scored 0.100 to 0.105. Big meetings flatten into near-complete graphs, which are hopeless to search for cliques. That is a finding about cost, not about accuracy.

E4: real committee data. House: 1290 people, 341 committees. Senate: 282 people, 315 committees. We score against party labels. - Best ECS is about 0.5, from one or two groups covering everyone. That is the trivial score for a two-party split. - On the House with the large-edge variant, no setting passes the giant-group test, so the rule falls back to the default. - The rule gives no gain here. Say it as a limit. Party is a weak answer key for committees.

---

6. Limits

- Planted groups are disjoint, so overlapping communities are never tested.
- Two party labels are a weak answer key.
- The method leaves some people out, and ECS punishes that.
- The rule is a heuristic. Its giant-group test assumes groups of similar size.
- On real committee data the rule does not help, and for the small-edge variant it does worse than the default.

---

7. Twelve likely questions

1. Is ECS 1.0 suspicious? No. At noise 0 the planted groups are the only structure. With the right variant and a pure size, noise is filtered by size, so the score stays at 1.

2. Why not modularity? Modularity is for ordinary graphs and assumes each person is in one group. We want overlap and we want to avoid flattening. We also have the answer key, so comparing to it is more direct.

3. Why ECS and not NMI or accuracy? Those need every person in exactly one group. Our groupings overlap and leave people out. ECS handles both, and the paper uses it.

4. What is noise fraction 0.4, concretely? 200 noise meetings per size, 600 in total, against 1500 community meetings.

5. Why did the rule not help on real data? The party labels give only a trivial best score of about 0.5 from one big group. The rule hunts for several large groups, which the labels do not reward. On the House with the large-edge variant it fell back to the default.

6. Did you reproduce the paper? No. We re-implemented it and checked it in the paper's setting. We do not claim to match its figures. We use 30 runs, the paper uses 100.

7. Why drop pairs on real data? A pair with one person in the pile always joins in the small-edge variant. Pairs would glue everything. The paper drops them.

8. What would you do next? Test overlapping planted groups, use richer labels than two parties, and design a rule for groups of different sizes.

9. How long does it run? The meeting-based method finished every run. A full E2 map of 2275 jobs took about 2547 seconds on 40 processes. Clique percolation on the flattened graph often hit the 60 second limit. We did not do a formal complexity study.

10. Why 30 runs, not 100? Time. The spread across runs was tiny (standard error 0.000 to 0.001 in E1), so more runs would not change the picture.

11. What is the oracle? The best settings for each hypergraph, chosen by looking at the answer key. It is a cheat that shows the ceiling. It is not something a user could do.

12. What does "the cut is not clean" mean? Community meetings and noise meetings share some sizes, so you cannot remove all noise by filtering on size. If one size is still pure, the method survives. If none is, it fails.

---

8. Slide-by-slide script (new deck, 20 slides)

1. Title. This is a re-implementation of a 2025 paper on finding communities from group data. We check it, we test where it breaks, and we add a rule for its settings. We do not claim to reproduce the paper's figures.

2. Groups are meetings, not pairs. Asha, Ben, Chen and Dev in one study session is one event. Flattening it into six pairs forgets that, and makes big meetings count too much. Many real groups, like committees, are meetings.

3. Communities, and two ways to assign them. A community is a set of people who meet together often. A partition gives each person one group. A cover allows overlap and left-out people. The method in this paper gives a cover.

4. Clique percolation in ordinary graphs. A clique is a set of people who all know each other. Two cliques are neighbours when they share all but one person. Chain neighbours and you get a community.

5. The paper's idea: chain meetings that overlap. Skip the pairs. Start from one meeting and add any meeting that overlaps the pile enough. Everyone in the pile is one community. Nothing is guessed. It is a rule.

6. Two versions of "overlaps enough". The large-edge variant keeps meetings of at least k people and joins them if they share I_abs people. The small-edge variant keeps meetings of at most K people and joins one if a fraction I_rel of it is already in. Use the first when small meetings are chatter, the second when huge meetings are meaningless.

7. What the paper leaves open. The four settings are left to the user. The

test is easy, because size alone separates community meetings from noise. And the comparison to clique percolation is by picture only.

8. The test bed. 500 people in 10 groups of 50. Community meetings come from inside one group. Noise meetings are drawn from everyone. At noise 0.4 there are 600 noise meetings against 1500 community meetings.

9. How we score a result. ECS asks, for each person, whether their group-mates are the same in the truth and in the result. 1 is identical and near 0 is unrelated. Left-out people count as a group of one, and overlap is allowed.

10. What we did. We rebuilt both variants with tests. E1 checks the paper's setting. E3 adds a rule for the settings. E2 and E4 are backup: an unclean size cut, and real committee data.

11. E1 figure, large-edge communities. Community meetings are 6 to 8 and noise is 3 to 5. Dark means good. The large-edge variant is 1.000 at every noise level. The small-edge variant drops from 1.000 to 0.424.

12. E1 figure, the mirror case. Community meetings are 3 to 5 and noise is 6 to 8. Now the small-edge variant stays at 1.000 and the large-edge variant drops to 0.206. The right variant stays perfect. The wrong one collapses.

13. Our rule for picking parameters without labels. Try every setting. Throw out settings where one group swallows the rest. Among the others, pick the one with the fewest, largest groups. If nothing survives, use the paper's default. Picking by coverage failed, because tiny fragments give high coverage.

14. E3 figure, rule against oracle and default. The large-edge rule scores 0.555 against a default of 0.292 and an oracle of 0.644. The small-edge rule scores 0.586 against 0.229 and 0.698. It hits the oracle on 46.6 and 55.4 percent of runs.

15. What we found. The right variant is exact when one meeting size is pure. With no pure size, both fail together. Clique percolation on the flattened graph is too costly on big meetings. Our rule beats the default and trails the oracle.

16. Limits. Planted groups are disjoint. Two party labels are weak. The

method leaves people out. The rule is a heuristic, and on real committees it gives no gain.

17. Conclusion. The method behaves as the paper says in its setting. It works when one size is pure and degrades when none is. Our rule is a cheap, label-free gain over the default, with a clear gap to the oracle.

18. Backup, E2 explanation. We let community and noise sizes overlap. With a clean cut or partial overlap, the matching variant still works. With full overlap, both variants fall to 0.601 at noise 0.1 and about 0.24 at 0.8. Clique percolation timed out in 278 of 350 runs.

19. Backup, E2 figure. Read it by layout. The partial-overlap panels stay near 1 for the matching variant. The full-overlap panel collapses for both. The missing clique cells are timeouts.

20. Backup, E4 committee data. House has 1290 people and 341 committees, Senate has 282 and 315. Best ECS against party is about 0.5 from one or two groups covering everyone. The rule gives no gain, and we state that as a limit.